

DSA Day 39: Merge Sort

Book: Data Structures Through C in Depth - Ch8 §8.12 · pp 434-444 | Code in Python

Overview

Divide the array, sort halves, merge. Stable, guaranteed $O(n \log n)$; ideal for linked lists.

Key concepts to master

- divide & merge
- stable
- extra $O(n)$ space

Python implementation

```
def merge_sort(a):
    if len(a) <= 1: return a
    m = len(a) // 2
    L = merge_sort(a[:m]); R = merge_sort(a[m:])
    out = []; i = j = 0
    while i < len(L) and j < len(R):
        out.append(L[i] if L[i] <= R[j] else R[j])
        if L[i] <= R[j]: i += 1
        else: j += 1
    return out + L[i:] + R[j:]
```

Complexity

$O(n \log n)$ time, $O(n)$ space.

Common pitfalls

- Losing stability with $<$ instead of $<=$
- Extra allocations

Practice

- Sort {8,3,5,1,9,2}
- Why merge sort suits linked lists

